Contents

1 Basic 1
1.1 default . . . 1
1.2 奇妙的東西 . . . 1
1.3 random . . . 1

2 Binary Search 1
2.1 二分搜 . . . 1
2.2 三分搜 . . . 1

3 Math 1
3.1 快速冪 . . . 1
3.2 Prime . . . 2

4 題單 2
4.1 鏈表 . . . 2

5 佛祖 3

1 Basic

1.1 default

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define idonthavegirlfriend ios::sync_with_stdio(0),cin
    .tie(0);
#define pb push_back
#define endl "\n"
#define all(v) (v).begin(),(v).end()
void solve(){

}
signed main(){
    idonthavegirlfriend
    int _=1;
    //cin>>_;
    while(_--){
        solve();
    }
}
```

1.2 奇妙的東西

```cpp

priority_queue<T> //由大到小
priority_queue<T,vector<T>,greater<T>> //由小到大
__builtin_popcountll// 二進位有幾個1(記得這是long long)
__builtin_clzll          // 左起第一個1之前0的個數
__builtin_parityll       // 1的個數的奇偶性
__builtin_mul_overflow(a,b,&h) // a*b是否溢位,h = a * b
    ;
__builtin_add_overflow(a,b,&h)
x>>=1 //右移最左邊補零
x<<=1 //左移最右邊補零
//註 : k 0-base
x&(1<<k) //取出 x 的第 k 位
if(x&(1<<k)) x = x ^ (1<<k) //把第 k 位歸零
x = x | (1<<k) //把第 k 位變成 1
x = ((1<<n) - 1) - x; //取 x 的補集合
//宇集 U = (1<<n) - 1
__builtin_popcount(x) //數 x 有幾個 1
```

1.3 random

```cpp
mt19937 mt(chrono::steady_clock::now().time_since_epoch
    ().count());
//mt19937_64 mt() -> return randnum
int randint(int l, int r){
    uniform_int_distribution<> dis(l, r); return dis(mt
        );
}
```

2 Binary Search

2.1 二分搜

```cpp
int bsearch_1(int l, int r)
{
    while (l < r)
    {
        int mid = l + r >> 1;
        if (check(mid)) r = mid;
        else l = mid + 1;
    }
    return l;
}
// ................oooooooooo
int bsearch_2(int l, int r)
{
    while (l < r)
    {
        int mid = l + r + 1 >> 1;
        if (check(mid)) l = mid;
        else r = mid - 1;
    }
    return l;
}
// oooooooooo...................
int m = *ranges::partition_point(views::iota(0LL,(int)1
    e9+9),[&](int a){
    return check(a) > k;
    });
//[begin,last)
//111111100000000000
//搜左邊數過來第一個 0
//都是 1 會回傳 last
int partitionpoint(int L,int R,function<bool(int)> chk)
    {
  int l = L,r = R-1;
  while(r - l > 10){
    int m = l + (r-l)/2;
    if(chk(m)) l = m;
    else r = m;
  }
    int m = l;
    while(m <= r){
        if(!chk(m)) break;
        ++m;
    }
  if(!chk(m)) return m;
  else return R;
}
//手工
```

2.2 三分搜

```cpp
int l = 1,r = 100;
while(l < r) {
    int lmid = l + (r - l) / 3; // l + 1/3區間大小
    int rmid = r - (r - l) / 3;  // r - 1/3區間大小
    lans = cal(lmid),rans = cal(rmid);
    // 求凹函數極小直
    if(lans <= rans) r = rmid - 1;
    else l = lmid + 1;
}
```

3 Math

3.1 快速冪

```cpp
int ksm(int x, int y) {
    int ans = 1;
    while (y) {
        if (y & 1)
            ans = ans * x;
        x = x * x;
        y >>= 1;
    }
    return ans;
}

// 矩陣乘法
vector<vector<int>> mul(const vector<vector<int>> &A,
                        const vector<vector<int>> &B) {
    if (A.empty() || B.empty() || A[0].empty() || B[0].
        empty()) { return {};}
    int m = A.size();
    int k = A[0].size();
    int n = B[0].size();
    vector<vector<int>> C(m, vector<int>(n, 0));
    for (int i = 0; i < m; i++) {
```

```
    for (int p = 0; p < k; p++) if (A[i][p]) {
    for (int j = 0; j < n; j++) {
    C[i][j] = (C[i][j] + 1LL * A[i][p] * B[p][j])%MOD;
}}}
    return C;
}

// 矩陣快速冪
vector<vector<int>> ksm(vector<vector<int>> base, int
    exp) {
    int n = base.size();
    vector<vector<int>> res(n, vector<int>(n, 0));

    for (int i = 0; i < n; i++) res[i][i] = 1;

    while (exp > 0) {
        if (exp & 1) res = mul(res, base);
        base = mul(base, base);
        exp >>= 1;
    }
    return res;
}
```

3.2 Prime

```
/* 12721, 13331, 14341, 75577, 123457, 222557, 556679
* 999983, 1097774749, 1076767633, 100102021, 999997771
* 1001010013, 1000512343, 987654361, 999991231
* 999888733, 98789101, 987777733, 999991921, 1010101333
* 1010102101, 1000000000039, 1000000000000037
* 2305843009213693951, 4611686018427387847
* 9223372036854775783, 18446744073709551557 */
int mu[ N ] , p_tbl[ N ];
vector<int> primes;
void sieve() {
  mu[ 1 ] = p_tbl[ 1 ] = 1;
  for( int i = 2 ; i < N ; i ++ ){
    if( !p_tbl[ i ] ){
      p_tbl[ i ] = i;
      primes.push_back( i );
      mu[ i ] = -1;
    }
    for( int p : primes ){
      int x = i * p;
      if( x >= M ) break;
      p_tbl[ x ] = p;
      mu[ x ] = -mu[ i ];
      if( i % p == 0 ){
        mu[ x ] = 0;
        break;
} } } }
vector<int> factor( int x ){
  vector<int> fac{ 1 };
  while( x > 1 ){
    int fn = SZ(fac), p = p_tbl[ x ], pos = 0;
    while( x % p == 0 ){
      x /= p;
      for( int i = 0 ; i < fn ; i ++ )
        fac.PB( fac[ pos ++ ] * p );
  } }
  return fac;
}
```

4 題單

4.1 鏈表

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define idonthavegirlfriend ios::sync_with_stdio(0),cin
    .tie(0);
#define pb push_back
#define yn(a) (a?"Yes\n":"No\n")
struct cmp{
        bool operator()(pair<int,int> a,pair<int,int> b
            ){
        if(a.first==b.first) return a.second>b.second;
        return a.first<b.first;
    }
};
signed main(){
    idonthavegirlfriend
    int n;
    cin>>n;
    vector<int> a(n);
    for(auto &c:a)cin>>c;
    vector<pair<pair<int,int>,pair<int,int>>> b;//many
        val pre nxt
    priority_queue<pair<int,int>,vector<pair<int,int>>,
        cmp> q;// many idx
    int cnt=1;
    int idx=0;
    for(int i=1;i<n;i++){
        if(a[i]==a[i-1]){
            cnt++;
        }
        else{
            b.pb({{cnt,a[i-1]},{idx-1,idx+1}});
            q.push({cnt,idx});
            cnt=1;
            idx++;
        }
    }
    b.pb({{cnt,a.back()},{idx-1,idx+1}});
    q.push({cnt,idx});
    cnt=1;
    idx++;
    int ans=0;
    vector<bool> vis(b.size(),false);
    while(!q.empty()){
        auto [x,id]=q.top();
        q.pop();
        if(vis[id]) continue;
        ans++;
        vis[id]=true;
        auto [pp,pn]=b[id].second;
        if(pp==-1 and pn==b.size()) continue;
        if (pp!=-1) b[pp].second.second=pn;
        if (pn!=b.size()) b[pn].second.first=pp;
        if(pn==b.size() and pp==-1) continue;
        if(pn!=b.size() and pp!=-1){
            if(b[pn].first.second==b[pp].first.second){
                int cnt=b[pn].first.first+b[pp].first.
                    first;
                vis[pn]=true;
                b[pp].first.first=cnt;
                b[pp].second.second=b[pn].second.second
                    ;
                if(pn!=b.size()){
                    int pnn=b[pn].second.second;
                    if(pnn!=b.size()){
                        b[pnn].second.first=pp;
                    }
                }
                q.push({cnt,pp});
            }
        }


    }
    cout<<ans;
}
```



5 佛祖

